Peer-based Content Verification Engine

Using Event-driven and Process-oriented Architectures in the Context of Content Verification

Event-driven and Process-oriented Architectures; Group 4: Evan Martino, Marco Birchler, Roman Babuk; Presentation: Assignment 1; Date: 16.04.2026

Table of contents

- Project Introduction
- Processes
- Architecture
- Trade-offs
- Demo & Q&A

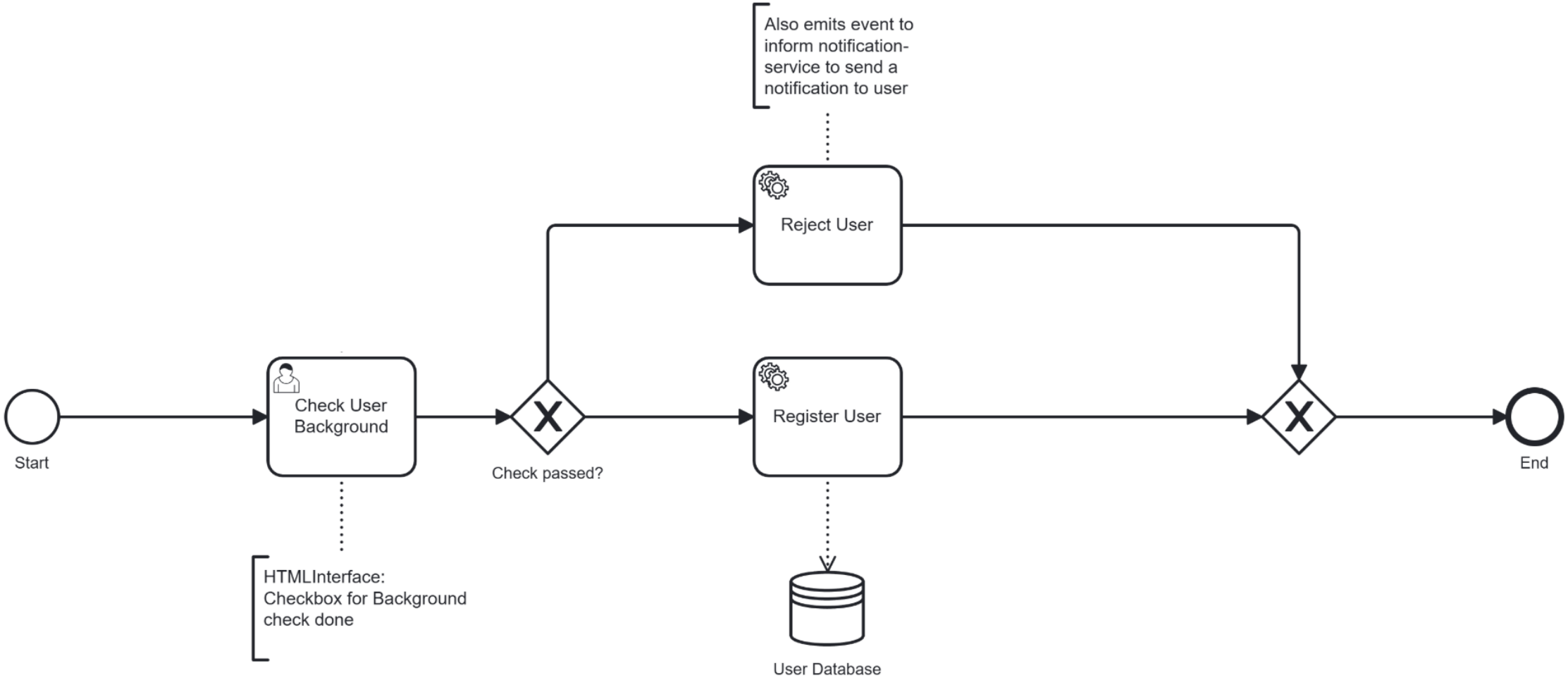
Project Introduction

- Content verification engine
- Verification signatures
- Based on peer-reviews
- Reporting system

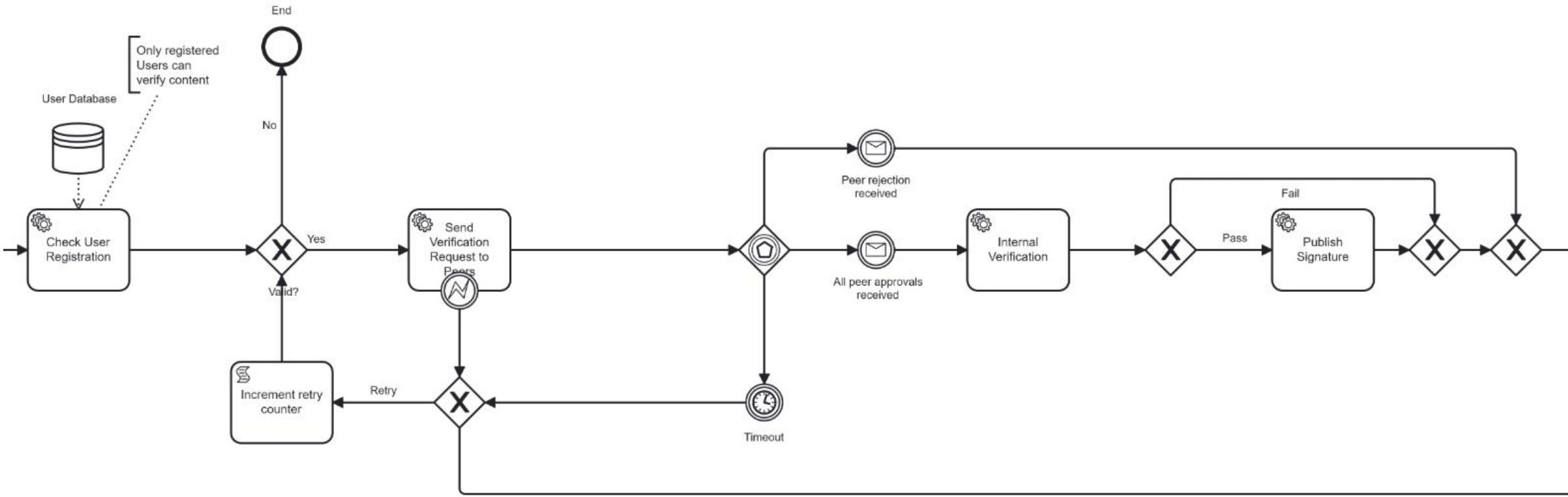
Table of contents

- Project Introduction
- Processes
- Architecture
- Trade-offs
- Demo & Q&A

Process: Register User



Process: Verify Content



Process: Report Content

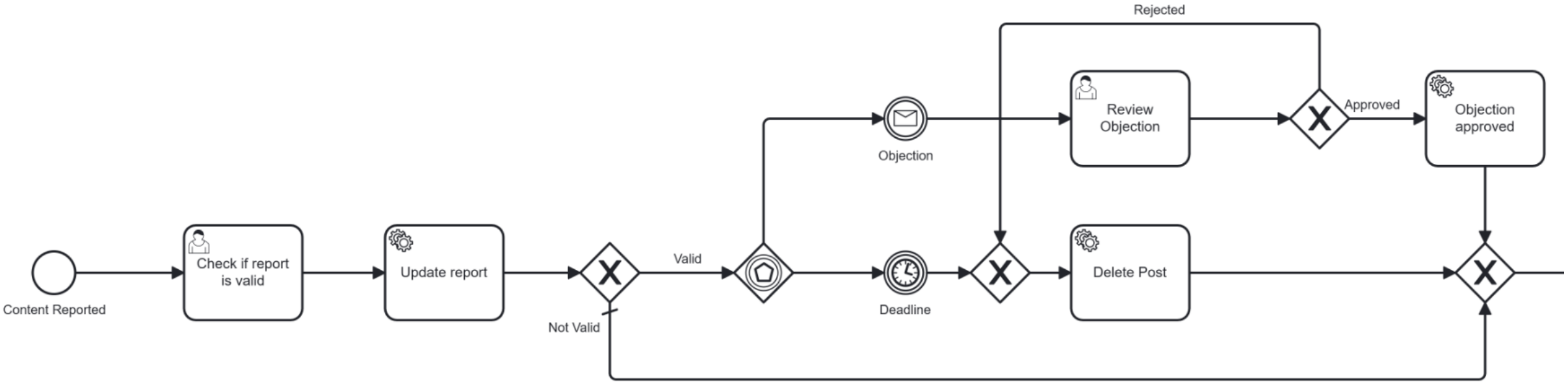


Table of contents

- Project Introduction
- Processes
- Architecture
- Trade-offs
- Demo & Q&A

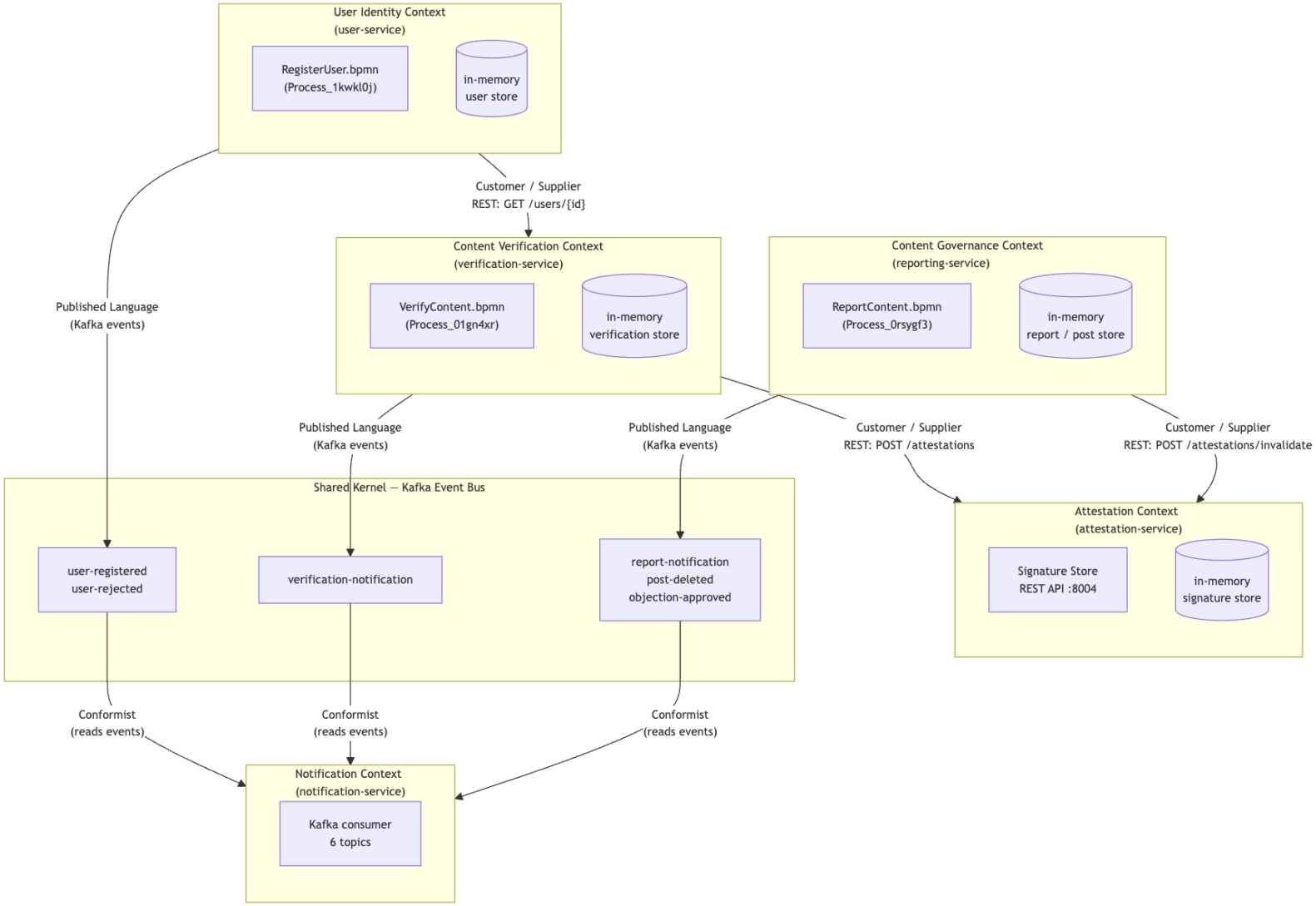
Architecture Overview

- Python Microservices
- Hybrid Orchestration/Choreography
- Event Notification over Kafka
- Stateful Retry
- Human Intervention

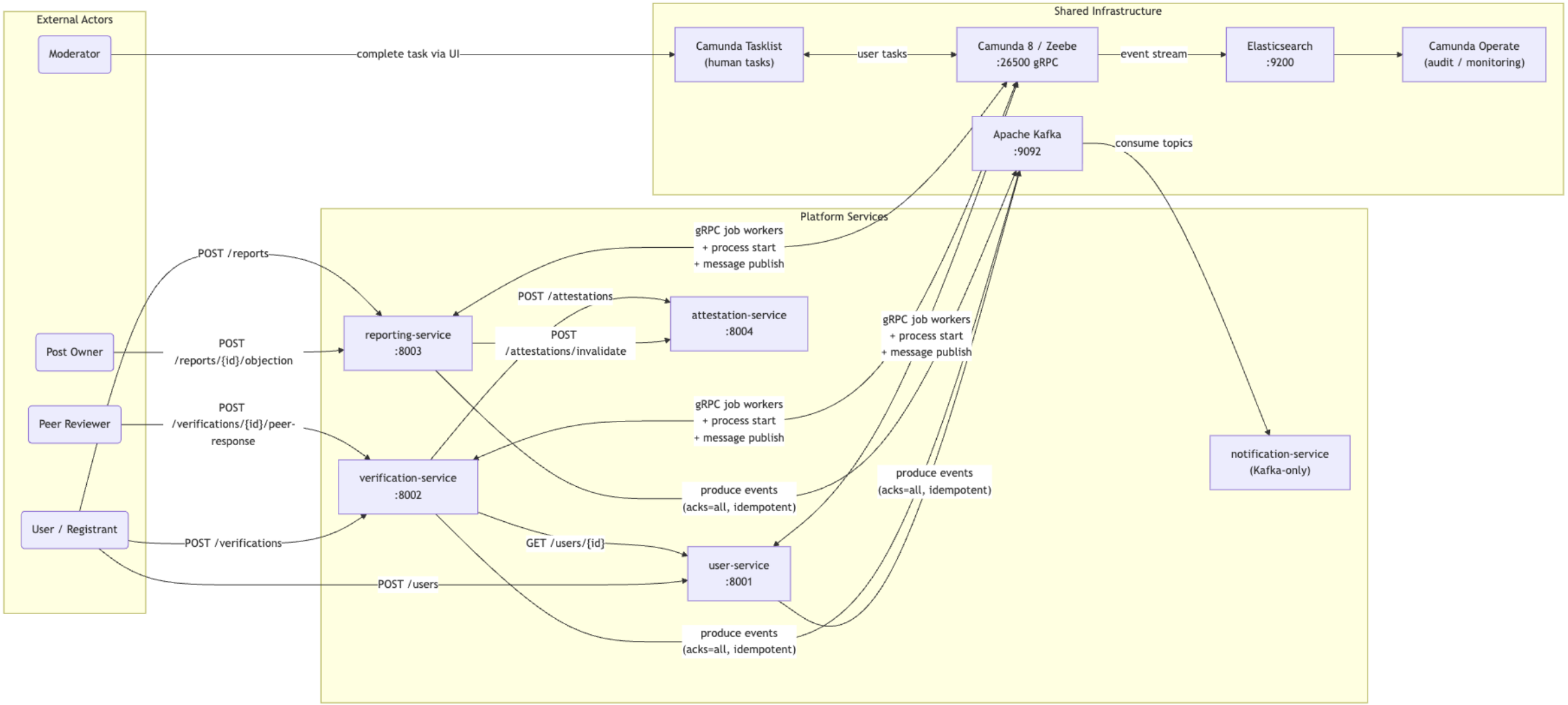
Architecture: Bounded Context via Python Microservices

- User Identity Context
- Content Verification Context
- Reporting Context
- Attestation Context
- Notification Context

Architecture: Bounded Context via Python Microservices



Architecture: Service Diagram



Architecture: Patterns from Lecture content

- Stateful Retry: Verification requests
- Human Intervention: Reviewing reports that cannot be resolved automaticaly

Table of contents

- Project Introduction
- Processes
- Architecture
- Trade-offs
- Demo & Q&A

Trade-off 1: Orchestration vs. Choreography

- Orchestration:
  - (+) control/visibility
  - (-) Camunda dependency

- Choreography:
  - (+) loose coupling/scalability
  - (-) harder global reasoning/debugging

➢ Hybrid Approach: Processes in Zeebe & Notifications via Kafka

Trade-off 2: Camunda 7 vs. Camunda 8

- Camunda 7:
  - (+) Easier implementation
  - (+) Less complexity
  - (-) Java based
  - (-) harder to scale

- Camunda 8:
  - (+) distributed, cloud-native Architecture
  - (+) Elasticsearch benefits our platform
  - (-) Additional complexity

➢ Camunda 8 fits our needs better

Trade-off 3: Eventual Consistency (Saga style)

- (+) independent service autonomy
- (-) separate services with local commits, async coordination

Trade-off 4: Human Intervention

- (+) safer for irreversible/high-risk decisions
- (-) slower throughput, requires moderators

Table of contents

- Project Introduction
- Processes
- Architecture
- Trade-offs
- Demo & Q&A

Demo & Q&A